

1. Describe how you implemented the program in detail. (annotation in program)

```
#define _GNU_SOURCE // 為了使用 sched_setaffinity 和 CPU_ZERO 等
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sched.h>
#include <time.h> // 為了 clock_gettime

// 全域變數，用於同步
pthread_barrier_t barrier;

// 結構體：用於傳遞參數給每個執行緒
typedef struct {
    int id;
    int policy; // SCHED_OTHER or SCHED_FIFO
    int priority; // 優先級 (FIFO 才有用)
    double time_wait;
} thread_info_t;

/**
 * @brief 忙碌等待 (Busy-Wait) 函式
 */
void busy_wait(double seconds) {
    struct timespec start, current;
    clock_gettime(CLOCK_THREAD_CPUTIME_ID, &start);

    while (1) {
        clock_gettime(CLOCK_THREAD_CPUTIME_ID, &current);
        double elapsed_sec = (current.tv_sec - start.tv_sec) +
            (current.tv_nsec - start.tv_nsec) / 1e9;

        if (elapsed_sec >= seconds) {
            break;
        }
    }
}

/**
 * @brief Worker 執行緒的進入點函式
 */
void *thread_func(void *arg) {
    thread_info_t *info = (thread_info_t *)arg;

    /* 1. 等待直到所有執行緒都準備好 (同步點) */
    pthread_barrier_wait(&barrier);

    /* 2. 執行任務 3 次 */
    for (int i = 0; i < 3; i++) {
        printf("Thread %d is running\n", info->id);

        /* 忙碌等待 <time_wait> 秒 */
        busy_wait(info->time_wait);
    }

    /* 3. 退出函式 */
    return NULL;
}
```

```

int main(int argc, char *argv[]) {
    int num_threads = 0;
    double time_wait = 0.0;
    char *policies_str = NULL;
    char *priorities_str = NULL;

    /* 1. 解析程式參數 (使用 getopt) */
    int opt;
    while ((opt = getopt(argc, argv, "n:t:s:p:")) != -1) {
        switch (opt) {
            case 'n':
                atoi(optarg) > 0 ? (num_threads = atoi(optarg)) : (num_threads = 0);
                break;
            case 't':
                atof(optarg) > 0 ? (time_wait = atof(optarg)) : (time_wait = 0.0);
                break;
            case 's':
                policies_str = optarg;
                break;
            case 'p':
                priorities_str = optarg;
                break;
            default:
                fprintf(stderr, "Usage: %s -n <num> -t <time> -s <policies> -p <priorities>\n", argv[0]);
                exit(EXIT_FAILURE);
        }
    }

    /* 檢查參數是否已提供 */
    if (num_threads == 0 || time_wait == 0.0 || policies_str == NULL || priorities_str == NULL) {
        fprintf(stderr, "Missing or invalid arguments.\n");
        fprintf(stderr, "Usage: %s -n <num> -t <time> -s <policies> -p <priorities>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    /* 2. 綁定 CPU: 將主執行緒 (以及它未來建立的所有子執行緒) 綁定到 CPU 0 */
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(0, &cpuset);
    if (sched_setaffinity(0, sizeof(cpu_set_t), &cpuset) == -1) {
        perror("sched_setaffinity");
        exit(EXIT_FAILURE);
    }

    /* 3. 初始化屏障 (Barrier) */
    pthread_barrier_init(&barrier, NULL, num_threads);

    /* 4. 準備執行緒屬性 */
    pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
    thread_info_t *thread_infos = malloc(num_threads * sizeof(thread_info_t));

    char *policies_str_dup = strdup(policies_str);
    char *priorities_str_dup = strdup(priorities_str);

    // 使用 strtok_r (可重入版本), 並宣告 saveptr 變數
    char *saveptr_policy;
    char *saveptr_priority;

    char *policy_token = strtok_r(policies_str_dup, ",", &saveptr_policy);
    char *priority_token = strtok_r(priorities_str_dup, ",", &saveptr_priority);

```

```

for (int i = 0; i < num_threads; i++) {

    if (policy_token == NULL || priority_token == NULL) {
        fprintf(stderr, "Error: Mismatch in number of policies/priorities and num_threads.\n");
        exit(EXIT_FAILURE);
    }

    thread_infos[i].id = i;
    thread_infos[i].time_wait = time_wait;

    // --- 設定排程策略 ---
    if (strcmp(policy_token, "NORMAL") == 0) {
        thread_infos[i].policy = SCHED_OTHER;
        thread_infos[i].priority = 0;
    } else if (strcmp(policy_token, "FIFO") == 0) {
        thread_infos[i].policy = SCHED_FIFO;
        thread_infos[i].priority = atoi(priority_token);
    } else {
        fprintf(stderr, "Error: Unknown policy %s for thread %d.\n", policy_token, i);
        exit(EXIT_FAILURE);
    }

    // 移動到下一個 token (使用 strtok_r)
    policy_token = strtok_r(NULL, ",", &saveptr_policy);
    priority_token = strtok_r(NULL, ",", &saveptr_priority);
}

```

```

0  /* 5. 建立並啟動所有 Worker 執行緒 */
1  for (int i = 0; i < num_threads; i++) {
2      pthread_attr_t attr;
3      struct sched_param param;
4
5      pthread_attr_init(&attr);
6
7      if (pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED) != 0) {
8          perror("pthread_attr_setinheritsched");
9      }
10
11     if (pthread_attr_setschedpolicy(&attr, thread_infos[i].policy) != 0) {
12         perror("pthread_attr_setschedpolicy");
13     }
14
15     param.sched_priority = thread_infos[i].priority;
16     if (pthread_attr_setschedparam(&attr, &param) != 0) {
17         perror("pthread_attr_setschedparam");
18     }
19
20     if (pthread_create(&threads[i], &attr, thread_func, &thread_infos[i]) != 0) {
21         perror("pthread_create");
22     }
23
24     pthread_attr_destroy(&attr);
25 }
26
27 /* 6. 等待所有執行緒完成 */
28 for (int i = 0; i < num_threads; i++) {
29     pthread_join(threads[i], NULL);
30 }
31
32 /* 7. 清理資源 */
33 pthread_barrier_destroy(&barrier);
34 free(threads);
35 free(thread_infos);
36 free(policies_str_dup);
37 free(priorities_str_dup);
38
39 return 0;
40 }

```

program passes all testcases

```

~ # ./sched_test.sh ./sched_demo ./sched_demo_a141064
Running testcase 1: ./sched_demo -n 1 -t 0.5 -s NORMAL -p -1
Result: Success!
Running testcase 2: ./sched_demo -n 2 -t 0.5 -s FIFO,FIFO -p 10,20
Result: Success!
Running testcase 3: ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Result: Success!
~ #

```

2. Describe the results of `./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30` and what causes that. (10%)

```

~ # ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Thread 2 is running
Thread 2 is running
Thread 2 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 0 is running
Thread 0 is running
Thread 0 is running

```

The output is strictly sequential, ordered by priority.

SCHED_FIFO (real-time) threads will *always* run before any SCHED_OTHER (normal) thread. Among SCHED_FIFO threads, the one with the highest priority number (30) always preempts (or runs before) the one with a lower priority (10).

SCHED_FIFO threads are not time-sliced. Once Thread 2 gets the CPU, it will run all three of its loops to completion *without interruption* before Thread 1 is ever allowed to start. The same applies to Thread 1 running before Thread 0.

3. Describe the results of `./sched_demo -n 4 -t 0.5 -s`

`NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30`, and what causes that. (10%)

```

~ # ./sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30
Thread 3 is running
Thread 3 is running
Thread 3 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running

```

Most of results are same with last question , but Thread 0 and Thread 2 are both "Normal" threads governed by the **Completely Fair Scheduler (CFS)**. Unlike FIFO, CFS is **fair** and uses **time-slicing**. It will not let Thread 0 run for its full 1.5s while Thread 2 starves. Instead, CFS **interleaves** them, giving a small time slice to Thread 0, then a small time slice to Thread 2, and so on. This "fair" sharing of the CPU causes their printf outputs to be mixed together until both are finished.

4.

```
/**
 * @brief 忙碌等待 (Busy-Wait) 函数
 */
void busy_wait(double seconds) {
    struct timespec start, current;
    clock_gettime(CLOCK_THREAD_CPUTIME_ID, &start);

    while (1) {
        clock_gettime(CLOCK_THREAD_CPUTIME_ID, &current);
        double elapsed_sec = (current.tv_sec - start.tv_sec) +
                               (current.tv_nsec - start.tv_nsec) / 1e9;

        if (elapsed_sec >= seconds) {
            break;
        }
    }
}
```

This function `busy_wait(double seconds)` implements a busy-wait delay. It continuously checks the elapsed CPU time of the current thread using `clock_gettime(CLOCK_THREAD_CPUTIME_ID, ...)` until the specified number of seconds has passed. The function measures the time difference between the start and current timestamps in seconds, and exits the loop once the elapsed time meets or exceeds the target.

5.

`kernel.sched_rt_runtime_us` is a **system-wide safety throttle** to prevent `SCHED_FIFO` (Real-Time, or RT) tasks from freezing the system.

- **What it does:** It defines the maximum CPU time that **all** RT tasks *combined* are allowed to use within a 1-second period (defined by `sched_rt_period_us`).
- Default (950000): Allows RT tasks to use 95% of the CPU. This reserves 5% of the CPU for normal tasks (like your shell or system services) so the system doesn't freeze.

*

Effect of Changing the Setting

- **950000 (Default):** If your RT tasks (like Thread 2 in the first test case) try to run for more than 0.95s, the kernel will **throttle** (pause) them to let normal tasks run. This would break your experiment.
- **500000 (50%):** RT tasks are heavily throttled and can only use 50% of the CPU.
- **1000000 (100%):** The throttle is off. An RT task with an infinite loop could **completely freeze the system**.